SYSTEM DESIGN

Design an Online Rental Platform (aka Airbnb)

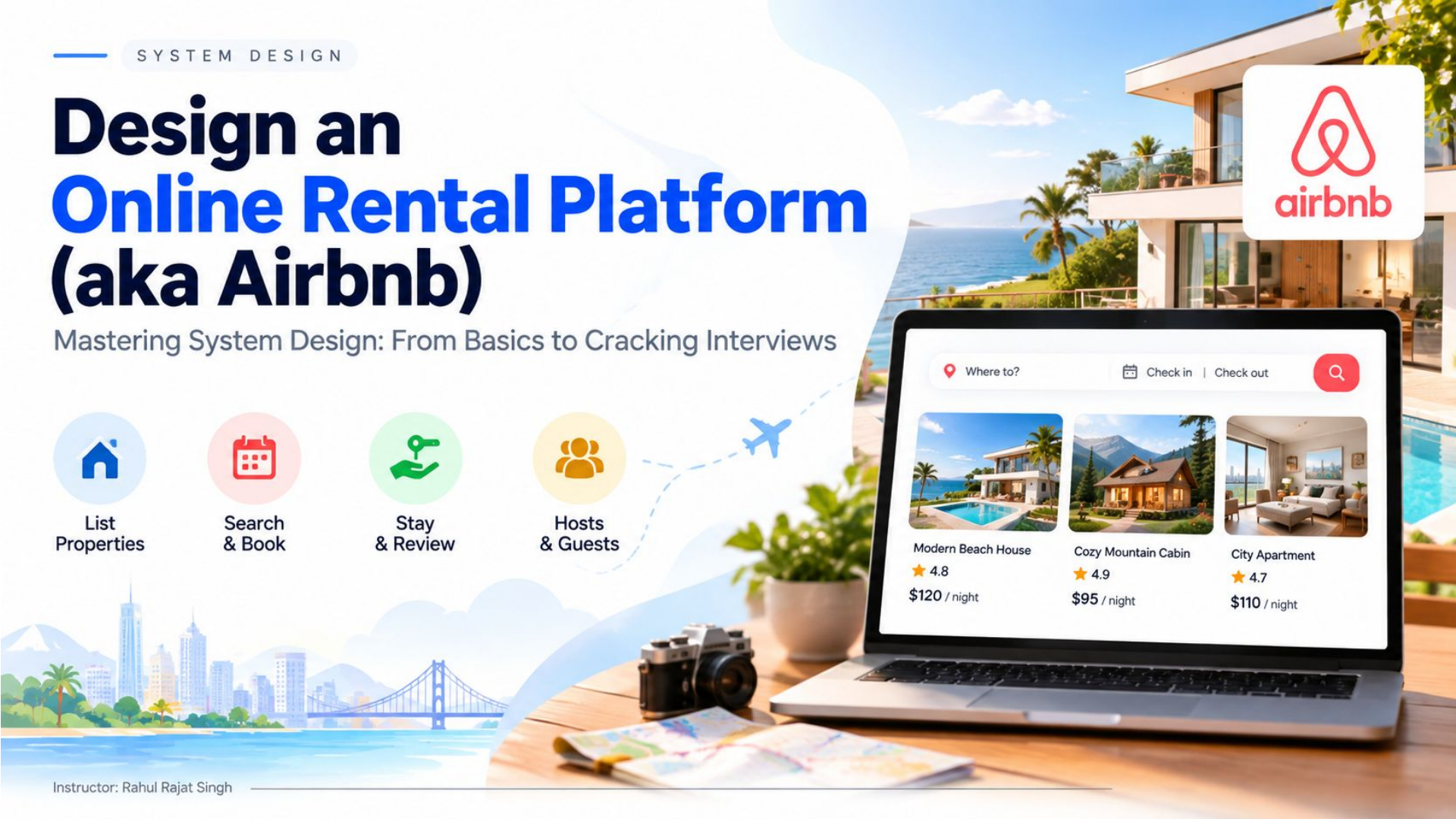
Mastering System Design: From Basics to Cracking Interviews

List Properties

Search & Book

Stay & Review

Hosts & Guests

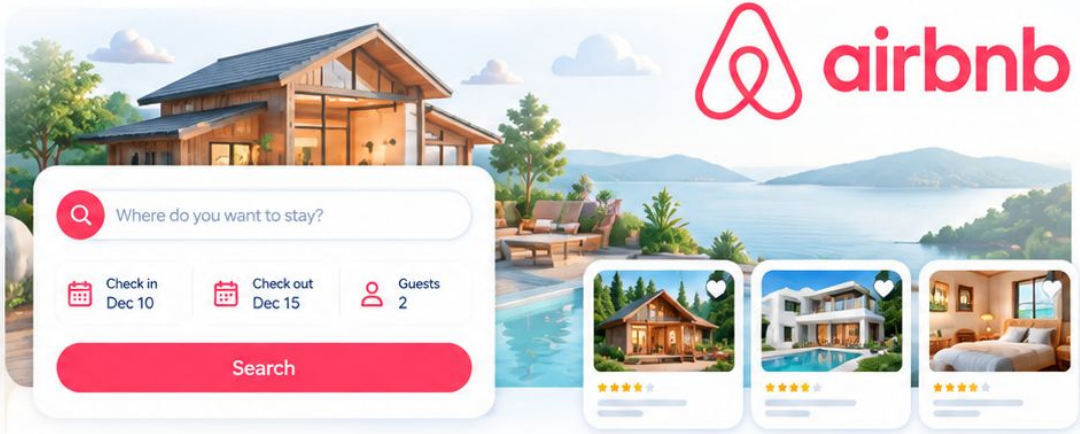


Instructor: Rahul Rajat Singh

Understanding the Problem & Key Actors

Problem Statement

"Design an Online Rental Platform where property owners can list accommodations and users can search, book, and pay for short-term stays."



Core Concepts

Listings
Properties, availability, pricing

Search
Search and filters

Bookings
Bookings and calendar synchronization

Payments
Payments and guest reviews

Media Uploads
Images and videos

Key Actors



Guest
Searches for and books accommondations



Host
Lists properties, manages availability and pricing



Admin
Moderates content, manages flagged users/listings



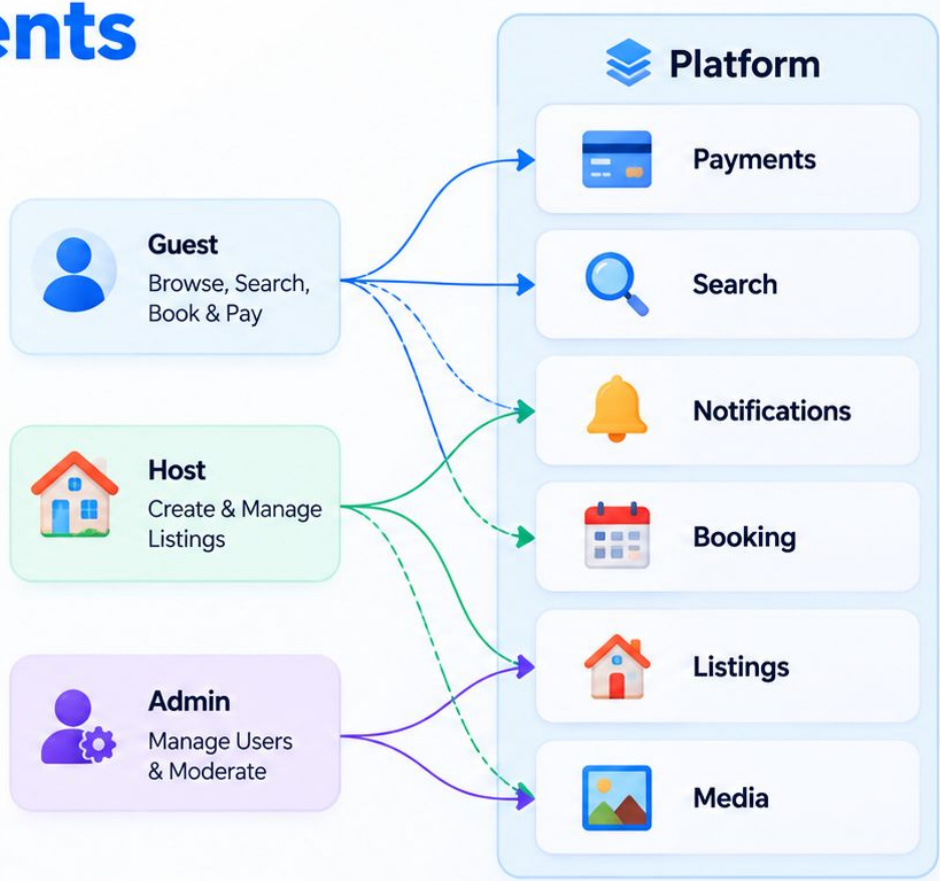
Payment Cateway
Processes secure transactions



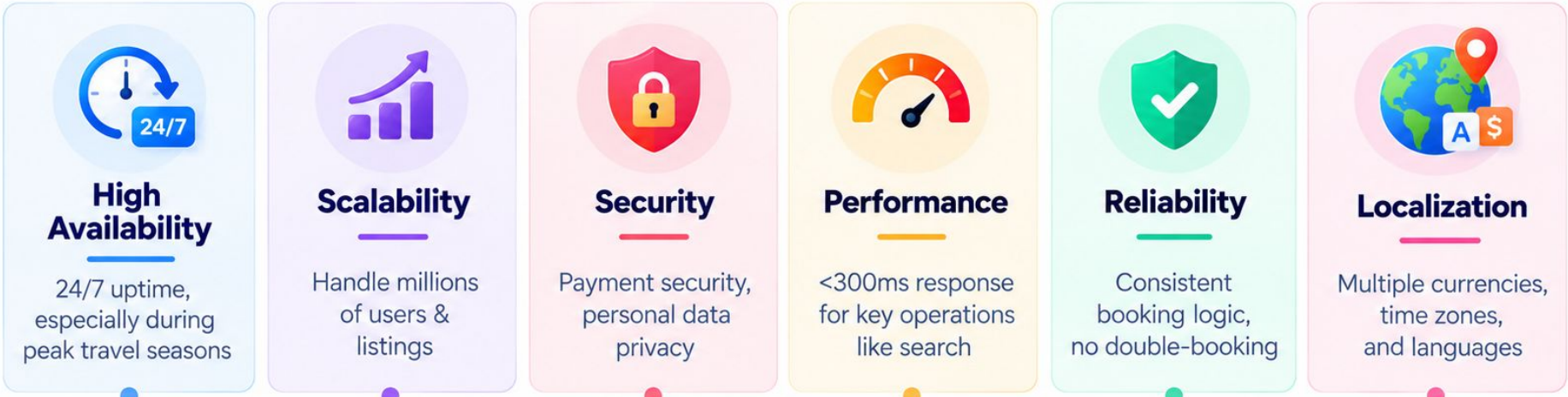
External Calendar
Syncs host calendars (Google Calendar, iCal, etc.)

Functional Requirements

1. **Account creation & login** (Host/Guest)
2. **Host can:**
   - Create & update listings
   - Upload media
   - Set pricing and availability
3. **Guest can:**
   - Search listings with filters
   - View listing details & reviews
   - Book listings and make payments
4. **Admins can manage users and moderate content**
5. **Calendar sync with external platforms**
6. **Notification system** (email, push)



Non-Functional Requirements



Assumptions and Constraints

1. Payments are handled via a **3rd-party gateway**
2. **Reviews** are moderated by the platform
3. Users must verify email before booking/listing
4. Media (images/videos) is uploaded to cloud object storage
5. Users use web and mobile apps (we need REST APIs)
6. Real-time search, but bookings can have slight delays (eventual consistency for availability)

- 3rd Party Payments
- Reviews Moderated
- Email Verification
- Cloud Object Storage
- Web + Mobile Apps REST APIs
- Eventual Consistency Availability

Online Rental Platform

Estimating Scale – Users, Listings, and Bookings

Daily Active Users (DAU)	Total active users per day (global)	~5 million globally
Concurrent Users	Peak concurrent users	~100k at peak
Listings	Total properties listed	50 million+ (growing daily)
Searches per day	Property searches per day	40–50 million
Bookings per day	Total bookings per day	~1 million
Media per listing	Images and videos per property	Avg. 10 images some videos
Payment transactions/day	Total payment transactions per day	~1 million (sync with bookings)



Implications

- **High volume of reads** (search, view listings)
- **Frequent writes** (bookings, availability updates)
- **Large media storage and delivery**
- **Real-time sync needed** (calendar, availability)

Data Size & Storage Needs

Rough Estimates

Listings DB ~50M listings × 5 KB	≈ 250 GB
Bookings DB ~1B records/year × 1 KB	≈ 1 TB+
Media storage 10 images × 1 MB × 50M	≈ 500 TB
User profiles + history ~5M DAU × 0.5 KB	≈ 2.5 GB/day

Hot Paths

- **Searching listings** (frequent reads)
- **Availability/calendar updates** (write-heavy)
- **Booking confirmation** (atomic write + payment + calendar update)

Cold Paths

- Reviews history
- User profile edits
- Admin moderation

Estimated Storage by Data Type

Media	Bookings	Listings	Profiles
~500 TB	~1+ TB / Year	~250 GB	~2.5 GB / Day

Identifying System Bottlenecks and Challenges

1. 

Search Service

- Handles very high query volume (QPS)
- Needs fast, filtered, geo-based search
- Requires scalable indexing and distributed querying

2. 

Availability Calendar

- Frequently updated due to bookings and sync with external calendars (iCal, Google Calendar, etc.)
- Requires consistency to avoid double-bookings
- Time zone management adds complexity

3. 

Booking System

- Atomic availability lock + payment + confirmation
- Needs to prevent race conditions during peak traffic
- May benefit from queues or transactions to ensure reliability

4. 

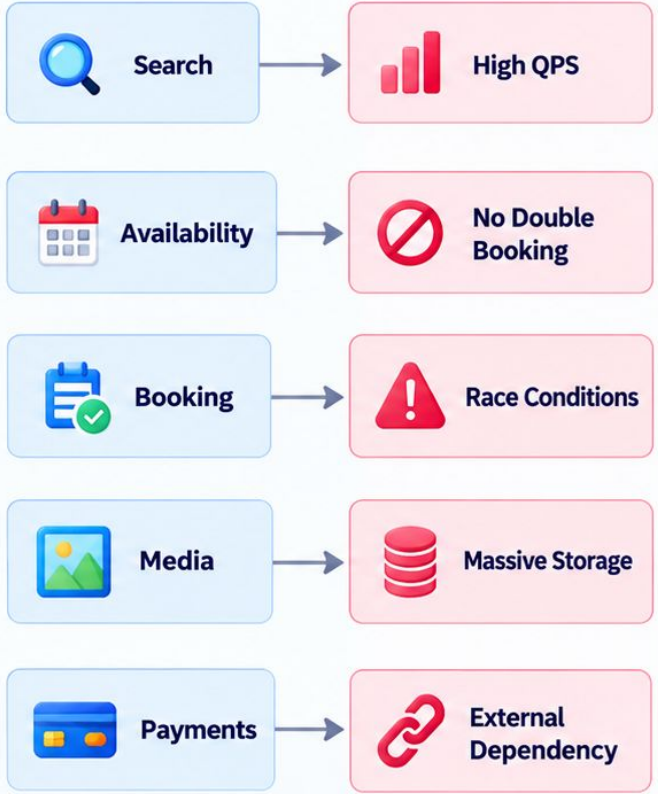
Media Storage & Delivery

- Huge storage needs (images/videos for millions of listings)
- High bandwidth and performance requirements for content delivery
- Needs CDN caching and object storage (e.g., S3, Azure Blob)

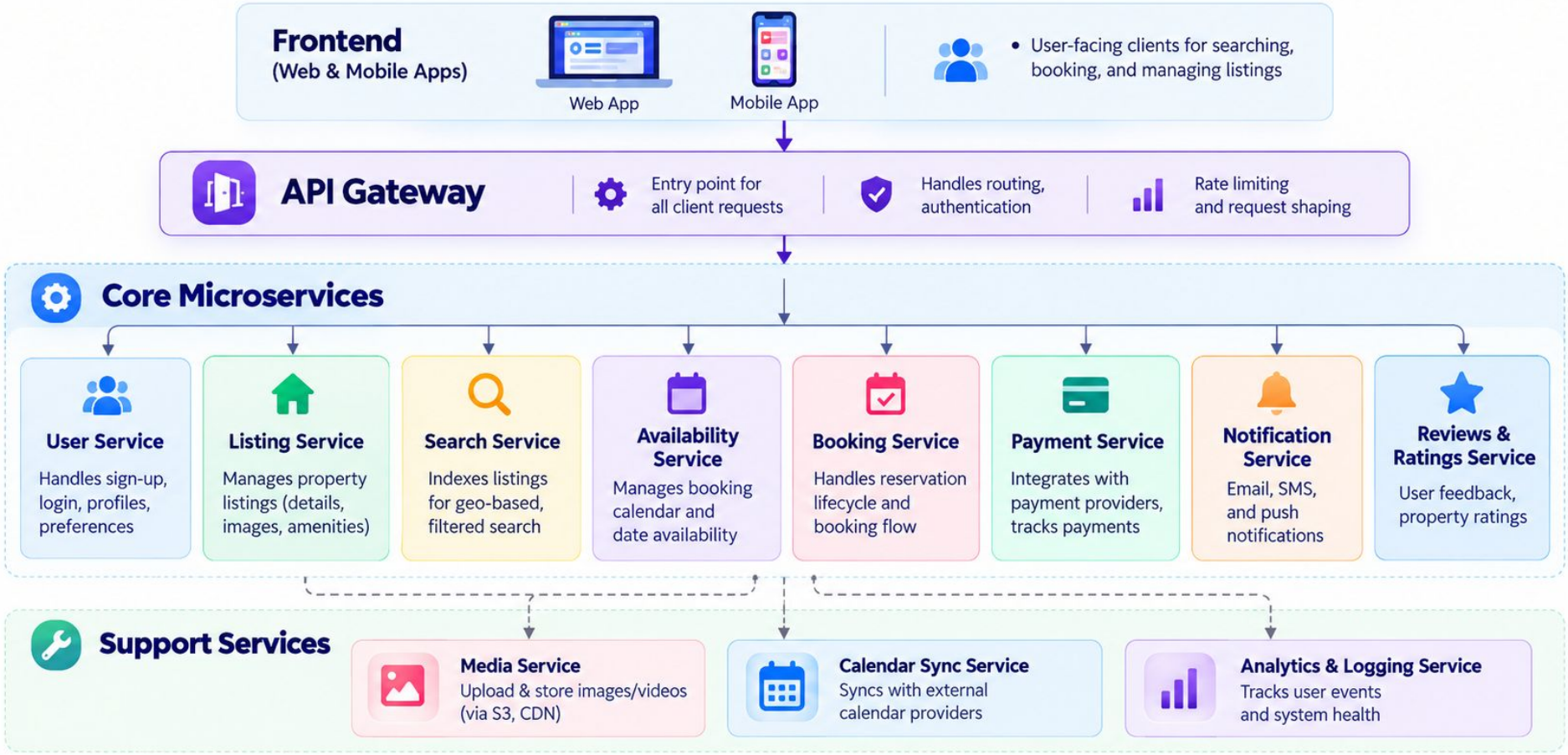
5. 

Payment Integration

- High dependency on third-party APIs (Stripe, PayPal, etc.)
- Must be fault-tolerant and secure (PCI compliance)
- Needs retry logic, logging, and fallbacks

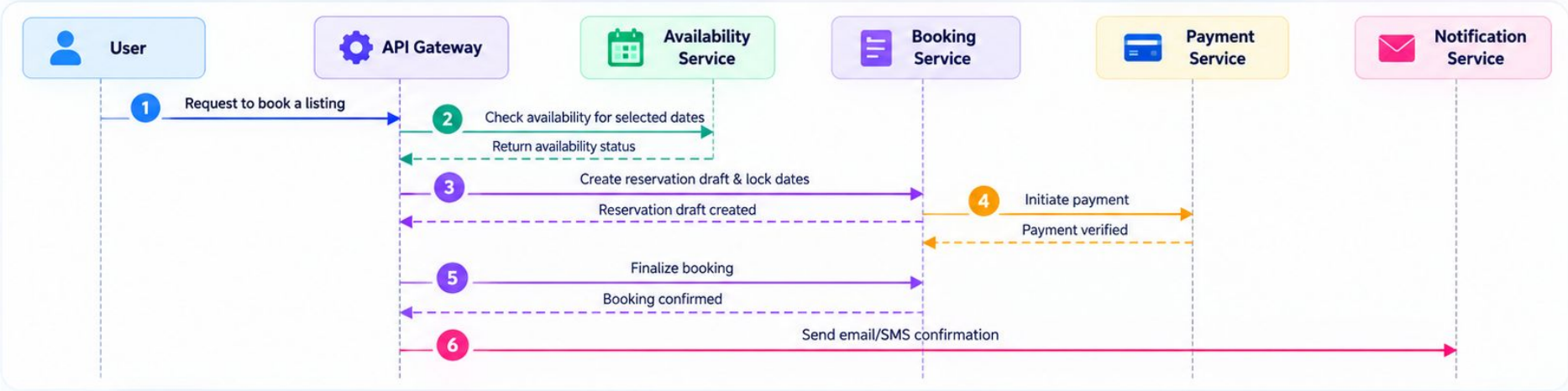


High-Level Architecture: Key Components



Service Interactions (Example: Booking Flow)

1. **User requests to book a listing**
2. **Availability Service** checks if dates are open
3. **Booking Service** locks availability, creates a reservation draft
4. **Payment Service** initiates and verifies payment
5. Upon success, **Booking Service** finalizes booking
6. **Notification Service** sends confirmation email/SMS



All interactions are **asynchronous** where possible (e.g., payment confirmation, notifications) using **message queues** (e.g., **RabbitMQ, Kafka**).

Communication Patterns & APIs

01 Sync Communication (REST / gRPC)

- Used for user interactions, search, listing fetch
- APIs exposed via API Gateway

User Interactions
Search
Listing Fetch

02 Async Communication (Message Queue / Event Bus)

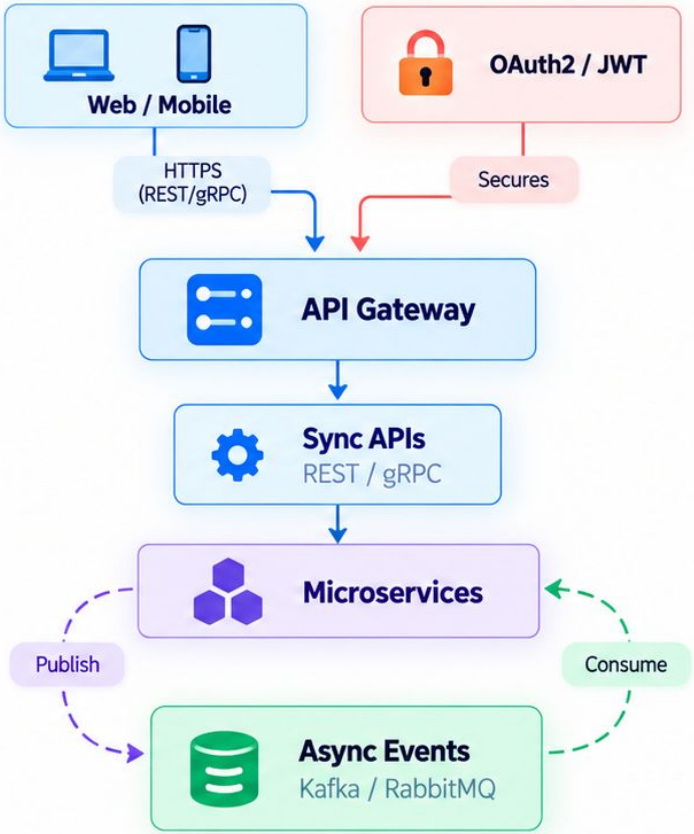
- Booking events (e.g., booking_created, booking_failed)
- Notifications, calendar sync, email dispatch
- Payments webhook events

Booking Events
Notifications
Email Dispatch
Payment Webhooks

03 Authentication & Authorization

- OAuth2 / JWT-based tokens
- Role-based access: guest, host, admin

OAuth2 / JWT
Roles: guest, host, admin



Data Storage & Indexing Strategy

1 Primary Datastores

Relational DB (PostgreSQL / MySQL)
For transactions, users & listing data.

NoSQL (MongoDB / DynamoDB)
For availability snapshots and reviews.

2 Search Index

Elasticsearch
For full-text and geo search on listings.

3 Caching Layers

Redis
For hot data (recent searches, popular listings).

CDN (Cloudflare / Akamai)
For images and static content.



Sample DB Schema for Online Rental Platform

Users
Platform users (hosts/guests)

Listings
Properties/items for rent

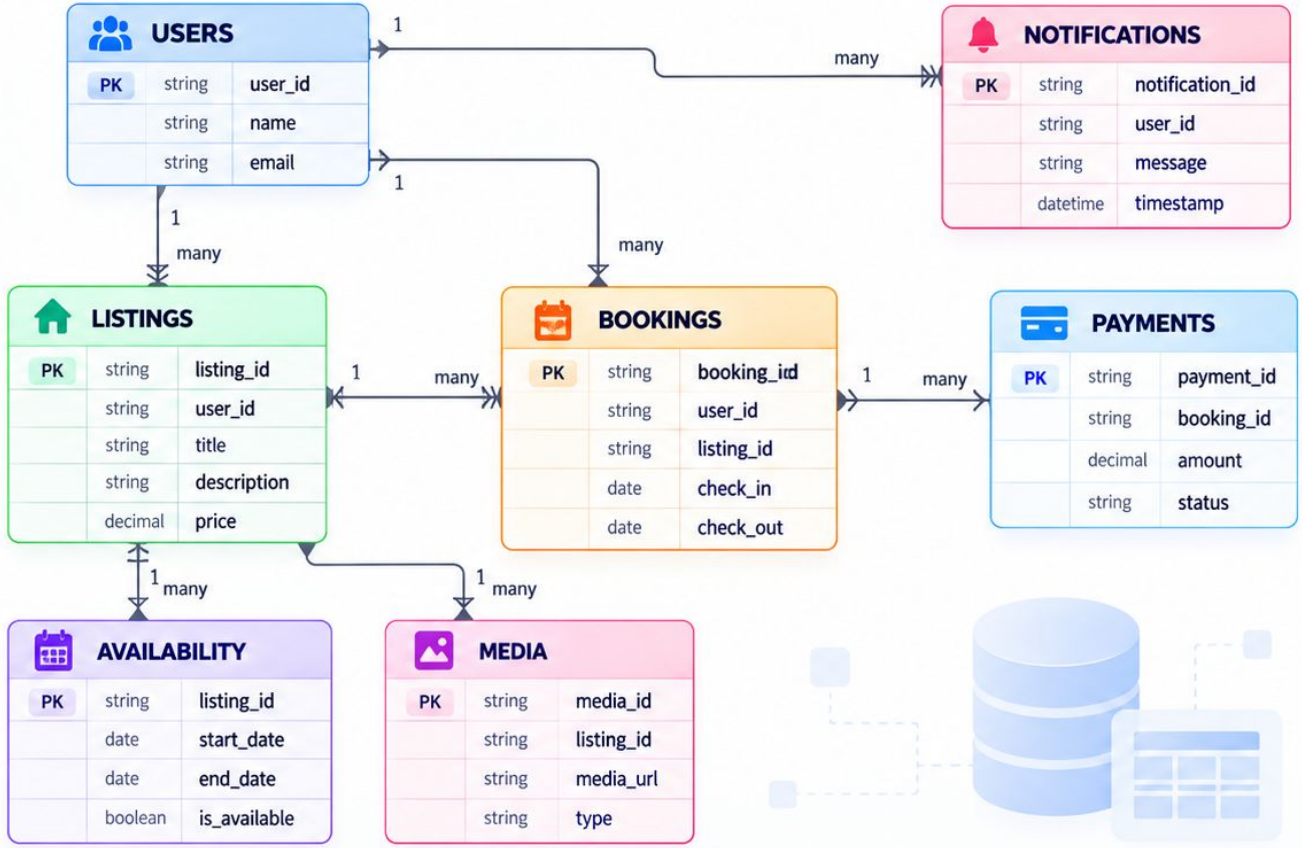
Availability
Available dates for a listing

Bookings
Confirmed reservations

Payments
Booking payments

Notifications
User notifications

Media
Listing images and media



Strategic Tech & Infra Decisions

1



Scalability

Use Azure for flexible scaling and horizontal scaling across multiple servers.

2



Redundancy & High Availability

Leverage load balancers and multi-region database replication for redundancy and availability.

3



Data Storage

Store transactional data in SQL and media assets in Azure Blob Storage for durability and cost-effectiveness.

4



Asynchronous Communication

Use RabbitMQ or Kafka for asynchronous tasks like payments and notifications to decouple services.

5



Caching

Integrate Redis for frequently accessed data to improve response time and reduce load.

6



Microservices Architecture

Adopt microservices to scale and maintain services (e.g., booking, payment, availability) independently.

7



Monitoring & Logging

Use Azure Monitor for health tracking and centralized logging (ElasticSearch) for real-time diagnostics.

The Final Design - Online Rental Platform

